BASES DE DATOS RELACIONALES

Módulo: Acceso a datos

Ciclo: Desarrollo de Aplicaciones Multiplataforma

Índice

1. Base de datos relacional
2. Protocolos de acceso a BD
    1. ODBC
    2. JDBC
3. Conector o driver
4. Diagrama de la clase JDBC
5. Ejemplos de uso
6. Transacciones
7. Patrón MVC

Base de datos relacional

- Presentan la información en tablas con filas y columnas
- Filas => ?
- Columnas => ?
- Claves:
  - ?
  - ?

Base de datos relacional

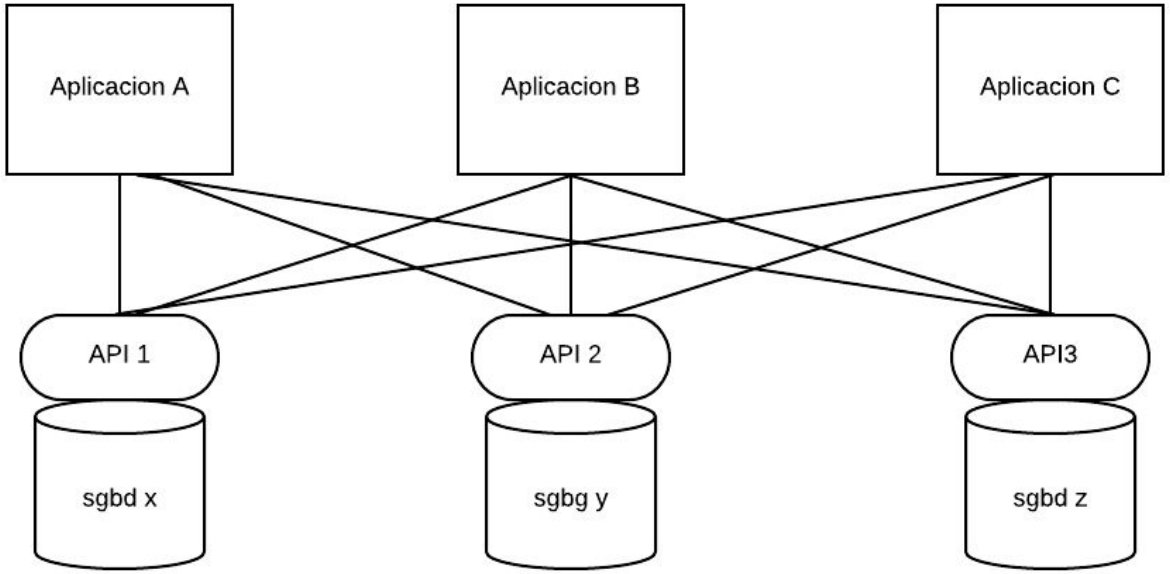
- Presentan la información en tablas con filas y columnas
- Filas => tupla
- Columnas => campos/atributos
- Claves:
  - ?
  - ?

Base de datos relacional

- Presentan la información en tablas con filas y columnas
- Filas => tupla
- Columnas => campos/atributos
- Claves:
  - Primaria identifican una tupla de forma unívoca
  - Foránea relación entre dos tablas.

Protocolos de acceso a BD

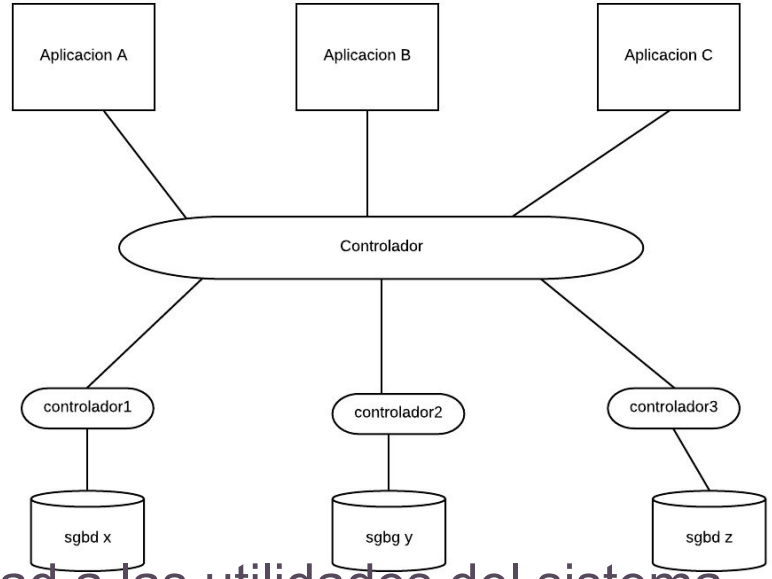
- En un inicio: cada SGBD implementaba soluciones propietarias



- Problemas:
  - Más lentos
  - Peor rendimiento

Protocolos de acceso a BD: ODBC

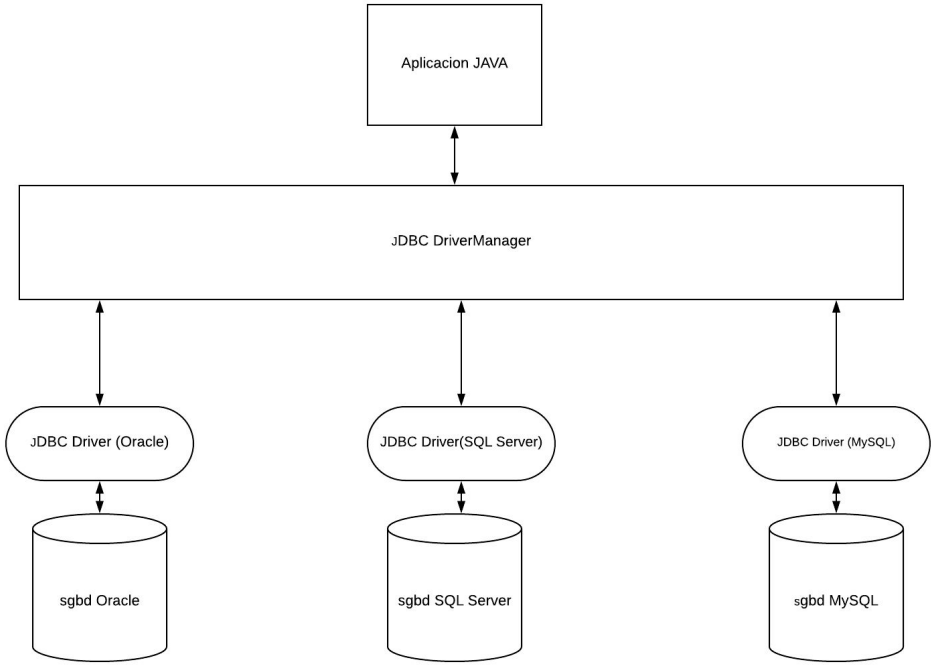
- Aparición ODBC (Open Database Conectivity)



- **Ventaja**:
  - Incorpora drivers de conectividad a las utilidades del sistema
  - Interoperabilidad: BD y lenguaje programación
- **Problema**: en Java no se introduce por su complejidad

Protocolos de acceso a BD: JDBC

- API similar a ODBC en cuanto a funcionalidad
- Diseñada para usarse con Java
- Simplifica el acceso a BD relacionales
- Proporciona un lenguaje:
  - Permite que las aplicaciones se comunican con el motor de BD
- Capa superior del modelo □ Java □ Aplicaciones portables



Protocolos de acceso a BD: JDBC

- No impone un lenguaje de comando común → Se permite usar la sintaxis de la BD en particular
- Clase responsable: **JDBC DriverManager**
  - Localiza el controlador JDBC necesario para conectarse a esa BD
  - Solicita conexión a la BD mediante URL:

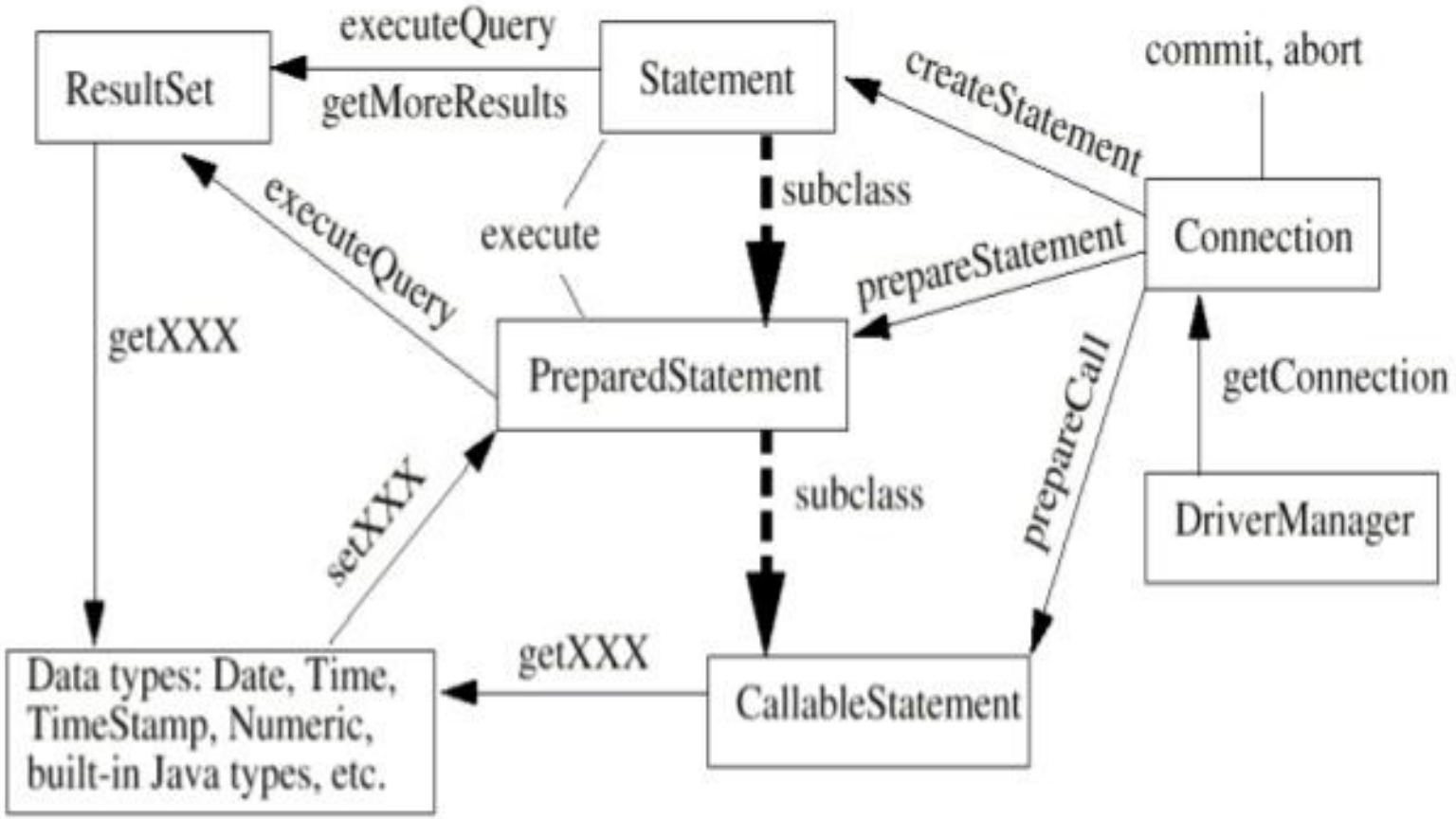
    jdbc:mysql://localhost:3306/nombreBD

Conector o driver

- Conjunto de clases encargadas de implementar las interfaces de la API y acceder a la BD
- Lo que viene siendo un fichero .jar
- Usaremos los paquetes **java.sql** →
- JDBC ofrecer las clases e interfaces para:
  - Establecer una conexión
  - Ejecutar consultas
  - Procesar los resultados

- **Connection**: permite gestionar una conexión
- **DriverManager**: permite obtener la conexión
- **[|Prepared|Callable]Statement**: crean la consulta
- **ResultSet**: maneja el resultado de las consultas

Diagrama de la clase JDBC



- **Statement**: ejecución consultas SQL normales
- **PreparedStatement**: ejecución de consultas dinámicas o parametrizadas
- **CalledStatement**: ejecución de procedimientos almacenados

Ejemplo de uso


```java
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class ConsultaSencilla {
    public static void main(String[] args) {
        Connection conexion;
        try {
            String urljdbc = "jdbc:oracle:thin:@localhost:1521:XE";
            //la base de datos esta en local y el servicio Oracle es XE se utiliza thin driver
            conexion = DriverManager.getConnection(urljdbc, "ejemplo", "ejemplo");
            Statement smt = conexion.createStatement();
            ResultSet rset = smt.executeQuery("select empno, ename, job from emp order by 1");
            while (rset.next())
                    System.out.println("empleado numero " + rset.getString(1) + " nombre " +
rset.getString(2) + " oficio " + rset.getString(3));
            conexion.close();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

- URL de conexión
- Constructor de conexión
- Creación consulta
- Ejecución de la consulta
- Obtención de los resultados
- Cierre de conexión

Ejemplo de uso: Insert, delete, update


```
Connection conexion = DriverManager.getConnection  ("jdbc:odbc:ORACLE-XE","scott", "tiger");
String deptno=args[0];
String sql= "DELETE FROM dept WHERE DEPTNO=?";
System.out.println(sql);
PreparedStatement sentencia = conexion.prepareStatement(sql);
sentencia.setInt(1,Integer.parseInt(deptno));
int filas = sentencia.executeUpdate();
System.out.println("Filas afectadas: " + filas);
sentencia.close();
    //Cerrar conexion
conexion.close();
```

- Definición consulta
- Marcamos posición del parámetro
- Creación de la consulta dinámica
- Ejecución + Obtención de los resultados
- Cierre de conexión

Ejemplo de uso: Procedimientos almacenados


```
// El procedimiento almacenado tendrá tres parámetros
CallableStatement prcProcedimientoAlmacenado = con.prepareCall("{ call insertaCliente(?, ?,?)

// cargar parametros en el procedimiento almacenado
prcProcedimientoAlmacenado.setInt("Cod_Cliente", 765);
prcProcedimientoAlmacenado.setString("Nombre", "Antonio Pérez") ;
prcProcedimientoAlmacenado.setString("Telefono", "950121314") ;

// ejecutar el procedimiento
prcProcedimientoAlmacenado.execute();
```

- Creación de la llamada
- Establecimiento de parámetros
- Ejecución

Transacciones

- Ejecución de un conjunto de consultas en su totalidad O se ejecutan todas o ninguna
- Permite garantizar:
  - Atomicidad
  - Consistencia
  - Aislamiento
  - Durabilidad
- Por defecto, viene Autocommit = true Cualquier consulta es ejecutada y persistida
- Deshabilitarlo setAutocommit(false)
- Ejecución de sentencias
- Finalización manual commit()

Transacciones


```
con.setAutoCommit(false);
updateSales = con.prepareStatement(updateString);
updateTotal = con.prepareStatement(updateStatement);
for (Map.Entry<String, Integer> e : salesForWeek.entrySet()) {
    updateSales.setInt(1, e.getValue().intValue());
    updateSales.setString(2, e.getKey());
    updateSales.executeUpdate();
    updateTotal.setInt(1, e.getValue().intValue());
    updateTotal.setString(2, e.getKey());
    updateTotal.executeUpdate();
    con.commit();
}
```

Patrón Singleton

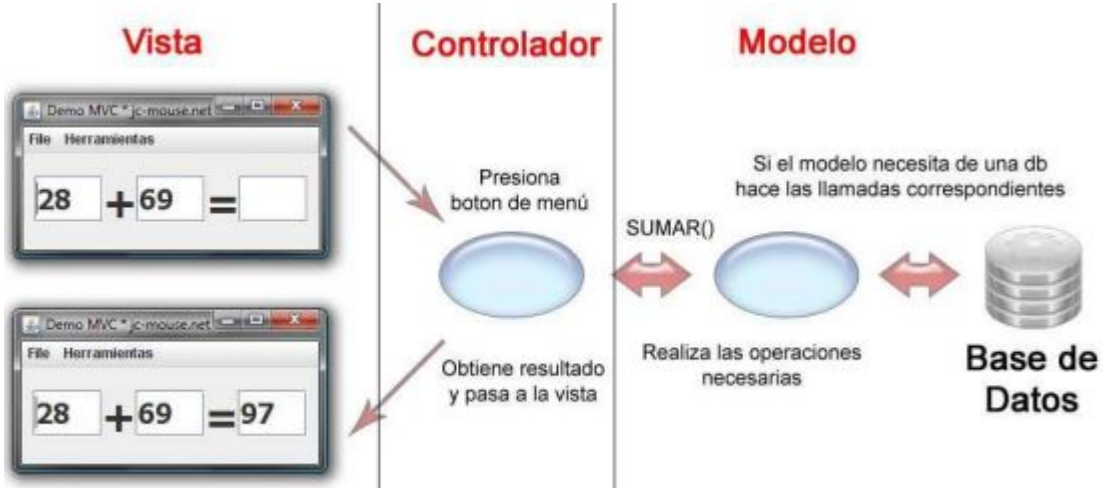
- Patrón de diseño que permite asegurar que una clase sólo tenga una única instancia.
- Resuelve dos problemas:
  - Garantiza una clase, una única instancia permite controlar el acceso al recurso compartido
  - Proporciona un punto de acceso global a la instancia funciona como variable global pero más segura
- Solución:
  - Hacer el constructor privado
  - Crear un método estático que actúe como constructor

Patrón Singleton


```java
public class Database{

    private static Connection connection;
    private final String usuario = "root";
    private final String clave = "abc123.";
    private final String url ="jdbc:mysql://localhost:3306/school";

    // El constructor del singleton siempre debe ser privado para evitar llamadas de construcción dir
    private Database(){
        try {
            this.connection = DriverManager.getConnection(url, usuario, clave);
        }catch(SQLException sqle) {
            System.out.println("Error al abrir la conexión");
        }
    }

    // El método estático que controla el acceso a la instancia
    // singleton.
    public static Connection getInstance(){
        if (Database.connection == null)
            Database.connection = new Database();
        return Database.connection;
    }
}
```

Patrón MVC

- Patrón modelo-vista-controlador de arquitectura de software
- Separa los datos de una aplicación:
  - Parte lógica del negocio
  - Parte interfaz de usuario
- **Modelo**: lógica de aplicación y sus datos
- **Vista**: parta de código con la que interactúa el usuario
- **Controlador**: comunica la vista y el modelo



Patrón DAO (Data Access Object)

- Patrón de diseño que separa la lógica de acceso a datos de los objetos de negocio encapsula lógica de acceso a datos del resto de aplicación.
- Problema:
  - Formato de la información puede variar según la fuente de los datos.
  - Mezcla lógica de negocio y lógica de los datos problemática a medida que aumenta el grado de complejidad del sistema.
  - Mayor problema si se utilizan más de una fuente de datos.
- Solución:
  - DAO proporciona métodos CRUD (Create, Read, Update, Delete)
  - Capa de negocio se preocupa de la lógica de negocio y usa DAO para acceder a los datos

Patrón DAO (Data Access Object)

```java
public interface DaoInquilinosInterfaz {
    public List <Inquilinos> inquilinosSel();
    public Inquilinos inquilinosGet(Integer id);
    public String inquilinosIns(Inquilinos inquilino);
    public String inquilinosUpd(Inquilinos inquilino);
    public String inquilinosDel(List<Integer> ids);
}
```

```java
public class DaoInquilinosImpl implements DaoInquilinosInterfaz {

    private final ConectaBD conectaDb;

    public DaoInquilinosImpl() {
        this.conectaDb = new ConectaBD();
    }

    @Override
    public List<Inquilinos> inquilinosSel() {
        // Código java de la consulta a la bd y gestiones pertinentes

    }

    @Override
    public List<Inquilinos> inquilinosSel() {
        // Código java de la consulta a la bd y gestiones pertinentes
    }
```